



Université de Lyon

Université Jean Monnet Saint-Étienne · Mines Saint-Étienne - Institut Mines Télécom

Master in Computer Science - Cyber-Physical and Social Systems: AI and IoT (CPS2)

Representation of Physical Quantities on the Semantic Web

M2 Internship Thesis submitted in partial fulfillment
of the requirements for the degree of
Master 2 in Computer Science

Irfan Ullah

Academic year 2024-2026

Mandatory research internship conducted at

Département Informatique et Systèmes Intelligents (ISI)
Institut Henri Fayol - Mines Saint-Étienne

1 February 2026 - 30 June 2026

Supervised by

Antoine Zimmermann

Professor
Head of ISI Department
Institut Henri Fayol
École des Mines de Saint-Étienne
LIMOS - UMR CNRS 6158
antoine.zimmermann@emse.fr

Maxime Lefrançois

Associate Professor
Institut Henri Fayol
École des Mines de Saint-Étienne
LIMOS - UMR CNRS 6158
maxime.lefrancois@emse.fr

June 29, 2026

Abstract

Physical quantities are fundamental to scientific, engineering, and medical data on the Semantic Web. The dominant approach is ontology-based: Quantities, Units, Dimensions and Data Types Ontologies (QUDT) and Ontology of units of Measure (OM) provide quantity kinds, unit hierarchies, dimensional vectors, and conversion factors. But four problems persist regardless of ontological richness. The most serious is computational inaccessibility: dimensional structure is encoded in the graph but never reaches the SPARQL Protocol and RDF Query Language (SPARQL) expression evaluator, so comparisons, ordering, and aggregation over mixed-unit data produce physically incomplete results without warning. The second is bounded coverage: derived units multiply combinatorial and no enumeration can be complete. The third is interoperability: independently developed ontologies assign different Internationalized Resource Identifiers (IRIs) to the same physical unit, requiring manual cross-ontology mappings. The fourth is verbosity: a single quantity value requires at least three triples.

The Linked Datatypes (LINDT) specification addresses all four problems at the datatype level through `cdt:ucum`, which encodes a complete quantity as a single typed literal using Unified Code for Units of Measure (UCUM). Because UCUM is a grammar rather than a list, coverage is unlimited by construction. The value space recognises dimensional equivalence natively, and operator semantics expose unit-aware equality, comparison, and arithmetic directly within SPARQL. The only existing implementation is a fork of Apache Jena and it is outdated, and no implementation exists for any other major Resource Description Framework (RDF) library. No general, reproducible procedure for building one has been documented.

This thesis addresses that gap. It presents a five-step methodology covering candidate library identification, extension point analysis, feasibility assessment, implementation, and validation. The feasibility step is an explicit go/no-go decision point before any code is written, and its output records not only whether each obligation can be met but through what mechanism and at what stability cost. A hard constraint frames the assessment: the implementation must leave the host library unmodified.

The methodology is validated through four working implementations across Apache Jena in Java, `rdflib.js` in TypeScript, `dotNetRDF` in C#, and `RDFLib` in Python, each paired with a UCUM backend suited to its ecosystem and accompanied by a test suite written against a single shared reference checklist. The central finding is that how far an extension can reach into a host from outside determines what the implementation can deliver. That reach is set by two factors: the host library's architecture and the host language.

The results confirm that the Custom Datatypes (CDT) approach is technically viable across heterogeneous programming ecosystems and provides a lightweight, standards-compatible path to unit-aware query evaluation on the Semantic Web.

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Antoine Zimmermann and Maxime Lefrançois, for their guidance throughout this internship. Their expertise in semantic web technologies and in the design of the CDT specification shaped this work in every Section. They were always available to discuss ideas, challenge assumptions, and point me in the right direction.

I thank the Département Informatique et Systèmes Intelligents (ISI) at Mines Saint-Étienne for welcoming me as an intern and providing a stimulating research environment.

I am also grateful to the Cyber Physical and Social Systems (CPS2) programme, jointly operated by Université Jean Monnet Saint-Étienne and Mines Saint-Étienne - Institut Mines Télécom, for the academic foundation that made this work possible.

This work would not have existed without the open-source community. The four implementations described in this thesis are built entirely on openly developed libraries and tools, including Apache Jena, rdfib.js, RDFLib, dotNetRDF, and the supporting libraries that make UCUM unit processing possible. The maintainers and contributors of these projects gave their time freely, and this research depended on that generosity at every step.

Finally, I am deeply grateful to my family for their unconditional support and encouragement throughout my studies. Their patience and belief in me made this work possible.

Contents

Abstract	i
Acknowledgements	ii
List of Abbreviations	ii
1 Introduction	1
1.1 Physical Quantities	1
1.2 The Problems	1
1.3 cdt:ucum as a Datatype-Level Solution	2
1.4 Prior Work and the Gap This Thesis Addresses:	2
2 Background	3
2.1 RDF	3
2.2 RDF Literals and Datatypes	3
2.3 SPARQL	4
3 Representation of Physical Quantities on the Semantic Web	4
3.1 Ontology-Based Approaches	4
3.2 Limitations of Ontology-Based Approaches	6
3.3 UCUM	7
3.4 The CDT Specification	7
4 Implementation Foundations	9
4.1 Implementations of Other Datatypes	9
4.2 RDF Libraries	9
4.3 UCUM Libraries	10
5 Methodology	11
5.1 Identify Candidate Libraries	11
5.2 Analyze Extension Points and APIs	12
5.3 Assess Feasibility	13
5.4 Implementation	13
5.5 Validation	14
6 Implementations and Findings	15
6.1 Identify Candidate Libraries	15
6.2 Analyze Extension Points and APIs	16
6.3 Assess Feasibility	17
6.4 Implementation	18
6.5 Validation	18
6.6 Findings Across the Cases	19
7 Conclusions	19

1. Introduction

1.1 Physical Quantities

Applications in many industry sectors rely on quantity values with units of measure, for sensor observations, actuation, design calculations and simulations, quantitative general knowledge, and clinical data. A temperature reading, for instance, is merely a bare number until it is paired with a specific unit such as degrees Celsius or Kelvin. The representation of such values is therefore critical: a numeric literal alone provides insufficient context, and associating each value correctly with its unit is a precondition for interoperable computation. To achieve this, the value and its unit must be encoded together, and any system that operates on them must respect the dimensional structure of the unit space. A precise foundation is therefore needed to define what quantities are, how they are classified, and how they combine.

The International Vocabulary of Metrology (VIM, JCGM 200:2012) makes the notion precise [1]. It defines a quantity as a property of a phenomenon, body, or substance whose magnitude can be expressed as a number and a reference, where a reference can be a measurement unit. The definition separates two ideas that everyday usage tends to blur. A quantity kind, such as length or temperature, is general and qualitative. A quantity value is specific: the height of one particular object, stated as a number multiplied by a unit. Dimensional analysis governs how quantity values combine. Two quantities can be compared or aggregated only if they share the same dimension. Two measurements of the same dimension expressed in different units denote the same quantity kind. A system that treats them distinct is dimensionally wrong. A system that operates across them without conversion is arithmetically wrong. Representing physical quantities correctly therefore means carrying magnitude and unit together and respecting the dimensional structure of the unit at every step.

In software, this discipline is enforced by unit-aware libraries. Pint in Python and Indriya in Java are representative examples, and comparable libraries exist in most programming languages. In these libraries a quantity is a magnitude paired with a unit. An operation between incommensurable quantities raises an error at runtime, while compatible quantities are converted automatically when combined. Their scope is computation inside a single application. What these libraries do not address is how quantities are stored, serialized, and queried across distributed systems. Hall and Kuster identify representational difficulties with quantities and units in digital systems in general and propose a central metrological register to address them [24].

On the Semantic Web the established answer is ontology-based. QUDT is an ontology, which grew out of a National Aeronautics and Space Administration (NASA) initiative, organises units within a hierarchy of quantity kinds and dimensions and records conversion factors. OM is also an ontology which provides a comparable unit vocabulary with finer-grained quantity kind hierarchies. Sensor, Observation, Sample, and Actuator (SOSA) and Semantic Sensor Network (SSN), published as a joint World Wide Web Consortium (W3C) and Open Geospatial Consortium (OGC) standard, model the observation itself and delegate the encoding of value and unit to QUDT or OM. Keil and Schindler [2] evaluated eight major unit ontologies and found coverage gaps and correctness issues across all of them.

1.2 The Problems

Consider a dataset that records distances from several sources. A construction survey reports in inches, a road network in miles, and a scientific instrument in metres. A SPARQL query asking which measurements exceed one kilometre ought to be straightforward. It is not. The query engine sees bare numbers next to unit IRIs and has no mechanism for treating them as commensurable quantities. Four problems lie behind this failure. Section 3 examines each in detail; this section states them in outline.

The first and most serious is computational inaccessibility. Ontologies such as QUDT and OM do encode dimensional structure and conversion factors, but that information never reaches the SPARQL

expression evaluator. Comparison, ordering, and aggregation operate on raw magnitudes, so queries across mixed units return physically incorrect results without warning. Lefrançois and Zimmermann [3] confirm that, they are not aware of any existing SPARQL engine provides unit-aware evaluation for QUDT or OM quantities. The only workaround is to make conversion factors explicit inside every query. At the time of that work, no such engine existed, and to the best of our knowledge that remains the case today.

The second is bounded coverage. An ontology enumerates its units one by one in a curated graph, yet the space of valid unit expressions has no natural boundary. Derived units multiply combinatorially: every pairing of a length unit with a time unit is a distinct speed unit, and the same holds for pressure, energy, and power. No enumeration can be complete.

The third is interoperability. Independently developed ontologies assign different IRIs to the same physical unit, and their vocabularies overlap only partially [2]. Combining datasets from different sources typically requires cross-ontology mappings, which must be built manually and may not exist for a given pair of ontologies.

The fourth is verbosity. Encoding one quantity value costs several triples. In a dataset with thousands of observations the overhead grows linearly with the data while adding no expressiveness.

1.3 cdt:ucum as a Datatype-Level Solution

The LINDT Custom Datatypes specification approaches these four problems from a different level: not the ontology, but the RDF datatype. It defines `cdt:ucum`, a datatype that encodes a complete quantity value as a single typed literal such as `"1.5 km"^^cdt:ucum`. The unit is part of the literal’s lexical form, written in the UCUM grammar. A single triple suffices; no blank node is required. The specification defines operator semantics for equality, comparison, and arithmetic, exposed through native SPARQL syntax, and its value space recognises dimensional equivalence, so two literals denoting the same physical quantity are value-equal regardless of the unit used to express them. Because UCUM is a grammar rather than a list, any unit the grammar can generate is valid. Coverage is unlimited by construction and independent of any curated vocabulary. None of this is achievable with ontology-based representations alone. Section 3 presents the specification and its formal datatype model in full.

1.4 Prior Work and the Gap This Thesis Addresses:

Lefrançois and Zimmermann [3] presented `cdt:ucum` at ESWC 2018 together with an implementation called `jena-ucum`. That implementation is a fork of the Apache Jena repository built against version 3, rather than a standalone extension. Using it means running the forked distribution instead of adding a dependency to a standard Jena installation. No other implementation exists for any other major RDF library, and no general procedure for producing one has been documented.

Research Question: This thesis addresses the following research question: How can the implementation of `cdt:ucum` in an RDF library be generalized into a procedure that is applicable and reproducible across different RDF libraries?

Contributions: This thesis makes three contributions. The first is a five-step methodology for implementing `cdt:ucum` in any RDF library. The second is four working implementations spanning Java (Apache Jena), TypeScript (`rdflib.js`), Python (`RDFLib`), and C# (`dotNetRDF`), each accompanied by a test suite that reflects what that library and UCUM backend combination can and cannot achieve. The third is an empirical analysis of how RDF library architecture and UCUM backend capabilities shape implementation strategy, documenting the limitations encountered in each case study and the patterns that recur across them.

Outline of Remaining Sections: Section 2 covers background on RDF, typed literals, and SPARQL. Section 3 surveys the representation of physical quantities on the Semantic Web, covering ontology-based approaches, the UCUM code system, and the CDT specification. Section 4 reviews prior custom datatype implementations and the candidate RDF and UCUM libraries. Section 5 presents the five-step methodology. Section 6 reports the four implementations and their findings. Section 7 concludes with a summary of contributions, stated limitations, and directions for future work.

2. Background

This section introduces the three building blocks the rest of the thesis depends on: the RDF data model, the typed literal and its datatype mechanism, and the SPARQL query language. The datatype mechanism in Section 2.2 deserves particular attention, since it is the extension point on which `cdt:ucum` is built.

2.1 RDF

The RDF is a W3C Recommendation for representing information in the Web [7]. Its data model is the triple: a subject, a predicate, and an object. A set of triples forms an RDF graph.

RDF 1.1 defines three kinds of node. An IRI is an absolute Unicode string conforming to Request for Comments (RFC) 3987, and two occurrences of the same IRI in a graph denote the same resource. A blank node has no global identifier; it asserts that some resource with the stated properties exists without assigning it a name. A literal represents a data value, and its structure is defined by a datatype IRI, covered in Section 2.2. A triple’s subject is an IRI or a blank node, its predicate is always an IRI, and its object may be any of the three.

RDF graphs can be serialized in several concrete syntaxes. Turtle uses compact prefix-based notation and is widely used for authoring and reading [16]. N-Triples encodes one triple per line with fully expanded IRIs, which makes it suitable for streaming and bulk processing. RDF/XML is the original serialization from the 2004 specification and remains widely supported. JSON for Linking Data (JSON-LD) encodes RDF as JavaScript Object Notation (JSON), binding prefixes to IRIs through a `@context` object, and is widely used in web-facing data publishing. This thesis uses Turtle for all RDF examples.

RDF itself imposes no vocabulary on what is being described. RDF Schema (RDFS) extends the model with constructs for defining classes, properties, and hierarchies between them [14]. Web Ontology Language (OWL) adds richer axioms covering class equivalence and disjointness, property characteristics, and cardinality [15]. Both vocabularies are themselves expressed as RDF triples.

2.2 RDF Literals and Datatypes

A literal consists of a lexical form and a datatype IRI. The lexical form is a Unicode string. The datatype IRI identifies the type and determines how the lexical form maps to a value. Language-tagged strings carry an additional language tag and use `rdf:langString` as their datatype IRI.

RDF 1.1 characterizes each datatype by three components. The lexical space is the set of Unicode strings that are valid representations of the type. The value space is the set of values the type can denote. The lexical-to-value mapping assigns each string in the lexical space a value in the value space. Two literals with the same datatype IRI are value-equal when their lexical forms map to the same value, even if the forms themselves differ. A literal whose lexical form falls outside the lexical space of its datatype is ill-typed. Such a literal is syntactically valid RDF, but it carries no value: it can be stored and matched by lexical form, yet it cannot participate in value comparison or arithmetic.

RDF 1.1 recognizes a set of Extensible Markup Language (XML) Schema built-in datatypes as its default type vocabulary, covering integers, decimals, floating-point numbers, strings, booleans, dates, and times, together with their numeric subtypes. Literals of these recognized types support equality testing and, for ordered types, comparison with the `<` and `>` operators. Beyond this set, RDF 1.1 defines two non-XML Schema Definition (XSD) built-in datatypes: `rdf:HTML` for typed HTML content and `rdf:XMLLiteral` for embedded XML.

The datatype mechanism is not closed to these types. Any IRI can serve as a datatype identifier. A processor that does not recognize a given datatype IRI treats its literals as uninterpreted: two such literals are value-equal if and only if they share the same lexical form and the same datatype IRI, and no further operations are defined over them. A processor that does recognize a custom datatype, whether through

built-in support or explicit registration, can equip it with a full value space and operator semantics. This is the mechanism that `cdt:ucum` exploits, and the one this thesis implements across four RDF libraries.

2.3 SPARQL

SPARQL 1.1 is the W3C Recommendation for querying RDF data [9]. It defines four query forms. `SELECT` returns variable bindings, `ASK` tests for pattern existence, `CONSTRUCT` builds a new RDF graph from query results, and `DESCRIBE` returns a description of a resource. A companion Update language handles graph insertion and deletion. The core construct is the triple pattern: it resembles an RDF triple but may carry a variable in any of the three positions. A set of triple patterns evaluated conjunctively forms a basic graph pattern. Matching a basic graph pattern finds all substitutions of variables with RDF terms that make every pattern consistent with the data, producing a solution sequence of variable-to-term bindings.

`FILTER` restricts a solution sequence to the solutions for which a given expression evaluates to true. `FILTER` evaluates its expression against the RDF terms that variables are bound to in the current solution. SPARQL 1.1 defines evaluation rules for recognized types: the `<` operator over two `xsd:integer` literals returns a boolean, and addition over `xsd:decimal` literals returns a typed numeric literal. For a literal with an unrecognized datatype, these operators raise a type error. The specification states that `FILTER` removes any solution for which the expression evaluates to false or raises an error. A type error therefore causes the solution to be silently excluded rather than reported as a query failure. This behaviour matters for everything that follows, since it determines how queries over custom-typed literals degrade in an engine without support for them.

`BIND` assigns the result of an expression to a new variable in the current solution, making the computed value available to subsequent patterns and expressions within the same query. Beyond comparison and arithmetic, SPARQL 1.1 provides a built-in function library covering numeric operations, string manipulation, date and time handling, and type testing.

`ORDER BY` sorts a solution sequence by one or more expressions. SPARQL 1.1 defines comparison for recognized types: numbers, strings, booleans, and `dateTime` values each have a specified ordering. For literals with unrecognized datatypes, the specification leaves the relative order undefined; it is implementation-dependent.

SPARQL 1.1 adds aggregate functions that operate over groups of solutions formed by `GROUP BY`. The built-in aggregates are `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, `GROUP_CONCAT`, and `SAMPLE`. For unit-aware quantity data, `SUM`, `MIN`, `MAX` and `AVG` are the most consequential. All apply arithmetic across group members and require operands of recognized numeric types. When a group member carries an unrecognized datatype, the arithmetic raises a type error and the aggregate result for that group is an error.

3. Representation of Physical Quantities on the Semantic Web

This part examines the representation problem introduced in Section 1. Section 3.1 presents the ontology-based approaches that constitute current practice. Section 3.2 analyses their limitations in detail, ordered from the most fundamental to the least. Section 3.3 introduces UCUM, the code system whose grammar resolves the coverage problem structurally. Section 3.4 presents the CDT specification, which builds on UCUM to address the problems at the datatype level.

3.1 Ontology-Based Approaches

The standard approach to representing physical quantities on the Semantic Web is ontology-based. Several ontologies have been developed for this purpose. Keil and Schindler evaluated eight of them:

Measurement Units Ontology (MUO), Extensible Observation Ontology (OBOE), OM, Quantities and Units (QU), QUDT, Semantic Web for Earth and Environmental Terminology (SWEET), Units Ontology (UO), and Wikidata [2]. QUDT and OM, examined in detail below, share one structural commitment: a single quantity value requires either a named individual or a blank node, together with several triples.

QUDT, originating from a NASA initiative, organises units within a hierarchy of quantity kinds and dimensions [4]. In QUDT, a quantity value is represented as a resource typed as `qudt:QuantityValue`. The magnitude is carried by `qudt:numericValue` as a numeric XSD literal, such as `xsd:double`, `xsd:decimal`, or `xsd:integer`; the unit is referenced by `qudt:unit` as a named individual. The following Turtle fragment illustrates the current QUDT encoding pattern:

```
@prefix sosa: <http://www.w3.org/ns/sosa/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix qudt: <http://qudt.org/schema/qudt/> .
@prefix unit: <http://qudt.org/vocab/unit/> .

<Observation/234534> a sosa:Observation ;
  rdfs:comment "Observation of the difference between the
    outside temperature and the inside temperature."@en ;
  sosa:hasFeatureOfInterest <apartment/134> ;
  sosa:hasResult [
    a qudt:QuantityValue ;
    qudt:unit unit:DEG_C ;
    qudt:numericValue "-29.9"^^xsd:double ] .
```

Four triples encode one quantity value. The magnitude and the unit are separated across two independent statements on the blank node, and no shared type binds them together. QUDT also encodes dimensional structure: each unit carries a `qudt:hasDimensionVector` property linking it to a vector of exponents over the fundamental dimensions, and a `qudt:conversionMultiplier` relating it to its International System of Units (SI) base equivalent [4].

OM, described by Rijgersberg, van Assem, and Top, follows a comparable reification pattern with finer-grained unit groupings [5]. A quantity value is an `om:Measure` blank node carrying a numeric value and a unit reference. An equivalent encoding in OM is:

```
@prefix sosa: <http://www.w3.org/ns/sosa/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix om: <http://www.ontology-of-units-of-measure.org/resource/om-2/> .

<Observation/234534> a sosa:Observation ;
  rdfs:comment "Observation of the difference between the
    outside temperature and the inside temperature."@en ;
  sosa:hasFeatureOfInterest <apartment/134> ;
  sosa:hasResult [
    a om:Measure ;
    om:hasUnit om:degreeCelsius ;
    om:hasNumericalValue "-29.9"^^xsd:double ] .
```

The structural commitment is the same: a blank node, a type triple, a magnitude triple, and a unit triple. OM provides more refined coverage than QUDT and finer-grained quantity kind hierarchies, but the reification pattern does not change. OM also encodes dimensional structure, through properties such as `SI_mass_dimension_exponent` and `SI_length_dimension_exponent` on dimension individuals [5].

SOSA and SSN, published as a joint W3C and OGC Recommendation, take an observation-centric approach [25]. A sensor observation is modelled as a `sosa:Observation` linked to a result via `sosa:hasResult`. SOSA and SSN do not prescribe how the numeric value and unit are encoded within the result; they are

typically combined with QUDT or OM for that purpose. The observation structure therefore adds further triples (as can be seen in the previous examples) on top of the quantity value pattern from whichever unit ontology is used.

Several further ontologies address the same problem space. MUO originated as a project to apply semantics in mobile environments, with its individuals automatically generated from UCUM [2]. OBOE takes an observation-centric approach developed primarily for ecological and scientific measurement scenarios [21]. QU is a showcase ontology based on the Object Management Group (OMG) SysML 1.2 Quantities, Units, Dimensions and Values (QUDV) specification and the United Nations Centre for Trade Facilitation and Electronic Business (UN/CEFACT) Recommendation 20 code list, published through the W3C Semantic Sensor Network Incubator Group [2]. SWEET covers a broad earth science vocabulary that includes units of measurement [22]. UO, part of the Open Biological and Biomedical Ontologies (OBO) Foundry, models units alongside the Phenotypic Quality Ontology (PATO) [2].

3.2 Limitations of Ontology-Based Approaches

The most fundamental problem is computational. QUDT represents dimensional structure through a vector of exponents over the fundamental dimensions: length, mass, time, electric current, temperature, amount of substance, and luminous intensity [4]. It provides `qudt:conversionMultiplier` per unit, relating each unit to its SI base equivalent. OM models dimensions similarly, with properties such as `SI_mass_dimension_exponent` and `SI_length_dimension_exponent` on dimension individuals [5]. This information is present in the graph and queryable in principle. It does not reach the SPARQL expression evaluator.

A FILTER expression evaluates against the RDF terms bound to query variables in the current solution, as established in Section 2.3. For a QUDT-encoded quantity, the term bound to the magnitude variable is a bare `xsd:double`, while the unit IRI is bound separately through a separate triple pattern. They are not a composite typed value. The dimensional relationships in the ontology are not carried into expression evaluation alongside the magnitude. A FILTER compares the raw `xsd:double` magnitude against 1.0 km with no awareness of units. A measurement of 500 metres passes `FILTER(500 > 1.0)` and is incorrectly included in the results, despite being shorter than one kilometre. Lefrançois and Zimmermann confirm that no existing RDF or SPARQL engine provides unit-aware evaluation for QUDT or OM quantities [3]. At the time of that work, no such engine existed, and to the best of our knowledge that remains the case today. Workarounds require making conversion factors explicit in every unit-aware query. When conversion information is retrieved from the ontology rather than hardcoded, additional triple patterns joining across the unit graph are needed, compounding the complexity of every unit-aware query.

The second problem is bounded coverage. Ontology-based unit vocabularies enumerate units as named individuals in a curated graph, and the scale of the enumeration required is larger than it appears. Length alone spans metric, customary, nautical, and astronomical systems. Derived units expand the space further: every pairing of a length unit with a time unit is a distinct speed unit, and the same holds for pressure, energy, and power. Each additional physical dimension multiplies the number of valid unit expressions by the number of units in that dimension. The set of valid unit expressions is unbounded.

Specific gaps have been documented across the evaluated ontologies. Keil and Schindler, evaluating eight major unit ontologies, found that coverage differed substantially between systems [2]. When a needed unit is absent, the extension mechanism requires defining it by its base units and conversion factors. Two teams independently extending the vocabulary for the same missing unit create two different IRIs for the same physical unit. The coverage gap thereby becomes an interoperability gap [3].

The third problem is interoperability. Multiple independently developed unit ontologies create a structural barrier at the IRI level. Keil and Schindler evaluated eight ontologies: MUO, OBOE, OM 1, OM 2, QU, QUDT, SWEET, and UO, together with the Wikidata corpus [2]. They found that these systems use different terms for the same concepts. The concept International Vocabulary of Metrology (VIM) calls kind of quantity appears as physical quality in MUO, quantity kind in QU and QUDT, and physical quantity in Wikidata. Unit individuals receive different IRIs across all eight systems, and mapping them revealed sparse `owl:sameAs` links and incorrect conversion factors in several ontologies [2]. Datasets encoded against different ontologies, or against different versions of the same ontology, cannot be queried uniformly without cross-ontology alignments. Those alignments are manually constructed and

are not guaranteed to be present.

The fourth problem is verbosity. The reification pattern shown in Section 3.1 requires at minimum four triples per quantity value regardless of content: one linking the subject to a blank node or named individual, one typing that resource, one carrying the magnitude, and one carrying the unit reference. Lefrançois and Zimmermann note that complex canonicalization mechanisms are needed to query these values uniformly [3]. In a sensor network generating thousands of observations, this overhead compounds proportionally with the dataset size.

3.3 UCUM

The UCUM is a code system developed by Gunther Schadow and Clem McDonald and distributed by the Regenstrief Institute [13]. Its purpose is to enable unambiguous machine-to-machine communication of quantities together with their units. Natural language unit names vary across languages, communities, and domains. UCUM expressions, by contrast, are case-sensitive character strings with a formal grammar, which makes them suitable for electronic data interchange without a separate disambiguation layer.

The grammar is compact and precisely defined. A unit expression is built from atomic unit symbols and metric prefixes using two operators, a period (.) for multiplication and a solidus (/) for division, with signed integer exponents written directly after the symbol. All symbols are case-sensitive: m denotes the metre and k the kilo prefix. Non-metric legacy units are enclosed in square brackets, such as [in_i] for the international inch. Dimensionless annotations are enclosed in curly braces. The grammar assigns each valid UCUM expression a precisely defined dimensional interpretation.

Coverage spans scientific, engineering, and business domains. SI base and derived units are included alongside metric prefixes, United States (US) customary units, and domain-specific units used in laboratory reporting, radiology, and pharmacology. UCUM has been adopted by Digital Imaging and Communications in Medicine (DICOM) [17], Logical Observation Identifiers Names and Codes (LOINC) [18], and Health Level Seven (HL7) [19], and is referenced by International Organization for Standardization (ISO) 11240:2012 [20].

UCUM defines a subset of its units as special units, whose conversion to base-unit form requires a non-linear function. Celsius (Cel) and Fahrenheit ([degF]) involve an additive offset relative to the thermodynamic temperature scale. pH involves a logarithmic transformation of hydrogen ion concentration. The bel (B) and its decimal subdivision the decibel are logarithmic ratios. These units are valid UCUM expressions, but their conversion semantics fall outside the multiplicative algebra.

3.4 The CDT Specification

Lefrançois and Zimmermann presented `cdt:ucum` at ESWC 2018 and formalized its specification in the LINDT Custom Datatypes document [3, 10]. A quantity value is encoded as a single typed literal whose unit is part of the lexical form rather than a separate triple. Operator semantics are defined for equality, comparison, and arithmetic, making them available through native SPARQL operators without custom function syntax.

The specification defines two general-purpose datatypes: `cdt:ucum`, with IRI <https://w3id.org/cdt/ucum>, and `cdt:ucumunit`, with IRI <https://w3id.org/cdt/ucumunit>. A quantity value encoded with `cdt:ucum` requires a single triple:

```
@prefix cdt: <https://w3id.org/cdt/> .  
BASE <http://example.org/>
```

```
<observation/235727> a sosa:Observation ;  
  sosa:hasFeatureOfInterest <house/134/kitchen> ;  
  sosa:observedProperty <electricConsumption> ;  
  sosa:madeBySensor <sensor/927> ;  
  sosa:hasSimpleResult "22.4 kWh"^^cdt:ucum .
```

Compared to the four-triple QUDT pattern in Section 3.1, the magnitude and unit are part of a single typed literal and no blank node is needed.

`cdt:ucumunit` encodes a bare UCUM unit expression without a magnitude. Its lexical space is the set of valid UCUM unit expressions. Its value space is the underlying set of the algebra of units defined in Section 3 of the UCUM specification. That algebra is formalized in §18, which defines the set U of units as the equivalence classes generated by the equality relation over UCUM expressions; two expressions belong to the same equivalence class when they denote the same unit, and $(U, \cdot)$ forms an Abelian group under unit multiplication. The lexical-to-value mapping sends each unit expression to its equivalence class in U . Two `cdt:ucumunit` literals are value-equal when they denote the same unit: `km.h-1`, `km/h`, and `(1000m)/(60min)` are all well-formed and share the same value.

`cdt:ucum` encodes a quantity value as a single literal. Its lexical space is the concatenation of the lexical form of an `xsd:decimal`, optionally followed by `e` or `E` and the lexical form of an `xsd:integer`, at least one whitespace character, and a valid UCUM unit expression. The value space is the set of pairs (v, u) where v is a real number and u is a product of UCUM base units raised to integer powers. This follows from §20 of the UCUM specification, which represents every proper unit as a pair of a scalar magnitude and a dimension vector over the base units; in the value pair, the magnitude is absorbed into v and u retains only the base-unit structure. The lexical-to-value mapping normalizes both components: the number $v = m \times 10^e$, where m is the decimal and e is the exponent (zero if absent); the unit $u = r \cdot b$, where r is the magnitude of the expressed unit relative to the base units b . The mapping assigns the lexical form to the pair $(v \times r, b)$.

Value equality follows directly from the mapping. The literal `"1 km"^^cdt:ucum` has decimal 1 and the unit `km`, whose magnitude relative to the metre is 1000. Its mapped value is $(1 \times 1000, \text{m}) = (1000, \text{m})$. The literal `"1000 m"^^cdt:ucum` has decimal 1000 and magnitude 1, giving $(1000 \times 1, \text{m}) = (1000, \text{m})$. The two pairs are identical, so the literals are value-equal.

The mapping is defined only for units that relate to their base units through a constant multiplicative factor. The UCUM specification calls these *proper units* and defines a separate category of *special units* (§21) for quantities measured on non-ratio scales, such as temperature and logarithmic quantities. A special unit is converted to and from its base unit by an arbitrary real function, not a constant scalar, so the `cdt:ucum` mapping has no way to handle it. The special units identified in Section 3.3 fall into this category: no constant factor converts Celsius to kelvin, or pH to moles per litre. An implementation built on this mapping therefore cannot correctly evaluate equality or comparison for special units. This is the principal known limitation of `cdt:ucum`, and it surfaces as expected test failures in each implementation examined in this thesis.

The specification defines operator semantics for four categories. The equality operator (`=`) tests value equality as described above. The comparison operators (`<` and `>`) compare two `cdt:ucum` literals by their normalized real value; they are defined only for literals of the same dimension and raise a type error when applied across incompatible dimensions. The arithmetic operators (`+` and `-`) add or subtract two commensurable literals, producing a result in the common base unit. The operators (`*` and `/`) multiply or divide literals of any dimensions, combining or cancelling dimensions accordingly, and both also accept a numeric scalar on either side. All four categories are exposed through native SPARQL operator syntax. The specification additionally defines the custom SPARQL function `cdt:sameDimension`, which takes two `cdt:ucum` arguments and returns an `xsd:boolean` indicating whether they are dimensionally commensurable.

A conforming implementation must satisfy the following obligations. It must correctly parse all lexical forms within the `cdt:ucum` lexical space and treat forms outside it as ill-typed. It must implement value equality using the lexical-to-value mapping. It must implement the comparison operators with the correct dimensional restriction. It must implement the arithmetic operators with their full type coverage, including scalar operands. It must register `cdt:sameDimension` as a callable SPARQL function. These obligations define the interface between the datatype and the host RDF library. Section 5 presents a methodology for satisfying them across different library and language environments.

4. Implementation Foundations

This section establishes the foundations the methodology builds on. Section 4.1 reviews prior custom datatype implementations, showing that the pattern `cdt:ucum` follows is established practice rather than an isolated design. Sections 4.2 and 4.3 survey the RDF libraries and the UCUM backends used in the four implementations. Together they supply the material that Step 1 of the methodology in Section 5 draws on.

4.1 Implementations of Other Datatypes

Section 2.2 established that any IRI can serve as a datatype identifier in RDF, and that a processor recognizing a custom datatype can equip it with a full value space and operator semantics. Three published systems show this mechanism at work in practice.

GeoSPARQL, published by the OGC, defines a vocabulary for representing and querying geospatial data in RDF [12]. It defines two OGC datatypes. The datatype `geo:wktLiteral` contains the Well-Known Text (WKT) serialization of a geometry object as defined by ISO 19125-1, while `geo:gmlLiteral` contains the Geography Markup Language (GML) serialization. A geometry is linked to its literal value through the property `geo:asWKT` or `geo:asGML`, and each literal embeds a coordinate reference system URI as a prefix within the lexical form. GeoSPARQL also defines a set of SPARQL extension functions for topological and geometric computation over these literals. The standard was updated as version 1.1 in 2024 [6].

Hartig et al. present `cdt:List` and `cdt:Map`, two custom datatypes for representing composite values as RDF literals [8]. A `cdt:List` literal encodes an ordered sequence; a `cdt:Map` literal encodes a set of key-value pairs. The approach introduces three SPARQL extensions: a FOLD aggregate that produces composite literals from a group of solutions, functions for accessing and manipulating composite literals within expressions, and an UNFOLD operator that decomposes a composite literal into one solution row per element.

Marciniak, Sowiński and Ganzha introduce a custom datatype for representing data tensors as RDF literals [11]. The motivation is the growing demand for integrating machine learning embeddings with knowledge graphs. The lexical form encodes tensor shape and values as a JSON string. The SPARQL extension defines 36 functions enabling the transformation, comparison, and concatenation of tensors, together with four aggregates.

All three cases apply the same pattern. A custom datatype IRI defines a value space and a lexical-to-value mapping for a domain-specific object, and SPARQL-level operators or functions are registered to expose computation over typed literals through native query syntax.

Each of the three systems solves a different problem: spatial geometry, composite data structures, and machine learning embeddings. What they share is the mechanism: a custom datatype IRI with a defined value space and operator semantics exposed through native SPARQL syntax. `cdt:ucum` applies the same mechanism to physical quantities, a domain where operator semantics must account for unit equivalence, dimensional compatibility, and the propagation of units through arithmetic.

4.2 RDF Libraries

RDF libraries provide the programmatic infrastructure for working with RDF data. Libraries differ in which features they include and in how accessible their internals are to extension code. Both differences matter for this thesis.

Apache Jena is an open-source Java framework for building Semantic Web and Linked Data applications.¹ Its SPARQL 1.1 engine, ARQ, supports federated queries. Jena also supports RDFS and OWL

¹<https://jena.apache.org>

reasoning through multiple configurable reasoners, Shapes Constraint Language (SHACL) validation, and a GeoSPARQL implementation. Its current release is version 6.x.x, distributed under the Apache License 2.0.

RDFLib is a Python library for working with RDF and the primary RDF library in the Python ecosystem. It provides parsers and serializers for the major RDF serializations and a full SPARQL 1.1 implementation for queries and update statements, together with a plugin architecture for extending stores, parsers, and serializers. Its current release is version 7.x.x, distributed under the Berkeley Software Distribution (BSD) 3-Clause license.

rdflib.js is a JavaScript RDF library for browsers and Node.js.² It reads and writes RDF/XML, Turtle, and N3, and reads RDF in Attributes (RDFa) and JSON-LD. It functions as a Read/Write Linked Data client, supporting Web Distributed Authoring and Versioning (WebDAV) and SPARQL/Update for interacting with remote data sources. rdflib.js does not ship a built-in SPARQL query engine; SPARQL interaction is limited to update operations against remote endpoints. Its current release is version 2.x.x, distributed under the MIT license.

dotNetRDF is a .NET library for parsing, managing, querying, and writing RDF.³ It provides a complete SPARQL 1.1 implementation through its built-in Leviathan in-memory query engine and supports the major RDF serialization formats. It also covers SHACL validation and a subset of OWL reasoning. Its current release is version 3.5.x, distributed under the MIT license.

Table 1 summarizes the four libraries.

Table 1: Summary of the four RDF libraries. GitHub statistics retrieved in June 2026.

Library	Language	First release	Stars	Forks	Contributors	Commits
Apache Jena	Java	2002	1.4k	697	144	13,020
RDFLib	Python	2002	2.5k	593	190	5,231
rdflib.js	JavaScript	~2012	593	145	47	1,687
dotNetRDF	C#	2011	322	99	37	5,016

4.3 UCUM Libraries

The libraries used across the four implementations fall into two categories. The first contains libraries that parse UCUM expressions and perform unit validation and conversion. The second contains general unit-arithmetic libraries: they maintain their own unit vocabulary and support quantity arithmetic, but do not parse UCUM expressions. The Python and Java implementations each pair one library from each category; the other two use a single library.

ucumvert is a Python package that parses UCUM unit strings and converts them to Pint quantities.⁴ It implements the full UCUM grammar using the Lark parser toolkit,⁵ generating the grammar directly from the official `ucum-essence.xml` file. Pint is the arithmetic backend: it represents quantities as numerical values paired with units, supports arithmetic with automatic unit conversion, and raises a dimensionality error for incommensurable quantities.⁶

ucum-lhc is a JavaScript library providing validation and conversion services for UCUM, developed by the Lister Hill National Center for Biomedical Communications at the US National Library of Medicine.⁷ It validates unit strings, converts numeric values between commensurable units, and finds units commensurable with a given expression.

Indriya is the reference implementation of the Units of Measurement Application Programming Interface (API) defined by Java Specification Request (JSR)-385,⁸ a Java Specification Request defining

²<https://github.com/linkedin/rdf-lib.js>

³<https://dotnetrdf.org>

⁴<https://github.com/dalito/ucumvert>

⁵<https://github.com/lark-parser/lark>

⁶<https://github.com/hgrecco/pint>

⁷<https://ucum.nlm.nih.gov/ucum-lhc>

⁸<https://jcp.org/en/jsr/detail?id=385>

standard interfaces for handling units and quantities.⁹ It covers unit compatibility checks, conversion between units, and arithmetic on quantities. The UCUM unit definitions are supplied separately by the uom-systems library, which plugs into Indriya’s unit framework. It is the arithmetic backend for the Apache Jena implementation in this thesis.

Fhir.Metrics is a C# library providing a UCUM implementation for the Fast Healthcare Interoperability Resources (FHIR) standard [19], developed and maintained by Firely.¹⁰ It reads the `ucum-essence.xml` data file and supports unit parsing, conversion, and quantity comparison.

Table 2 summarizes the libraries used across the four implementations.

Table 2: Summary of the UCUM libraries. GitHub statistics retrieved in June 2026.

Library	Language	First release	Stars	Forks	Contributors	Commits
pint	Python	2012	2.7k	515	225	2933
ucumvert	Python	2024	15	3	3	103
Indriya	Java	2017	138	48	19	1,405
ucum-lhc	JavaScript	2016	63	17	7	990
uom-systems	Java	2017	38	20	13	671
Fhir.Metrics	C#	2014	24	15	12	209

5. Methodology

This section presents a five-step procedure for implementing `cdt:ucum` in an RDF library. The procedure is general. It applies to any programming language, any RDF library that supports custom datatypes, and any UCUM backend available in that language. It assumes no particular library architecture and no particular level of SPARQL support. Both vary across environments, and the procedure is designed to make that variation explicit rather than to hide it.

Each step states its goal, prescribes the actions the implementer must carry out, and names the output it produces. The output of each step is the input of the next. This chaining is what makes the procedure repeatable: a second implementer starting from the same specification and the same candidate libraries should reach the same decisions for the same reasons.

5.1 Identify Candidate Libraries

The goal of this step is to identify two separate things: an RDF library to host the implementation, and a UCUM backend to handle unit parsing and arithmetic. The two searches are independent and proceed in parallel.

For the RDF library, the implementer must list the libraries available in the target language and subject each to a surface check covering three properties. The first is active maintenance, which is the floor rather than the whole of it: the library’s age, contributor base, and breadth of adoption count alongside it, and a library may lack a major feature and still be the natural host if the community has settled on it. The second is SPARQL coverage; a library without a SPARQL engine is not discarded outright, but its available query mechanisms must be documented, because the operator obligations may still be deliverable in reframed form. The third is the presence of a custom datatype mechanism; without it, the registration obligation cannot be met.

For the UCUM backend, the implementer must identify libraries that can parse UCUM codes, convert between units, and perform arithmetic. Since UCUM-native libraries are scarce in most languages, the search must also include general unit-arithmetic libraries; these cannot serve alone but can pair with a

⁹<https://github.com/unitsofmeasurement/indriya>

¹⁰<https://github.com/FirelyTeam/Fhir.Metrics>

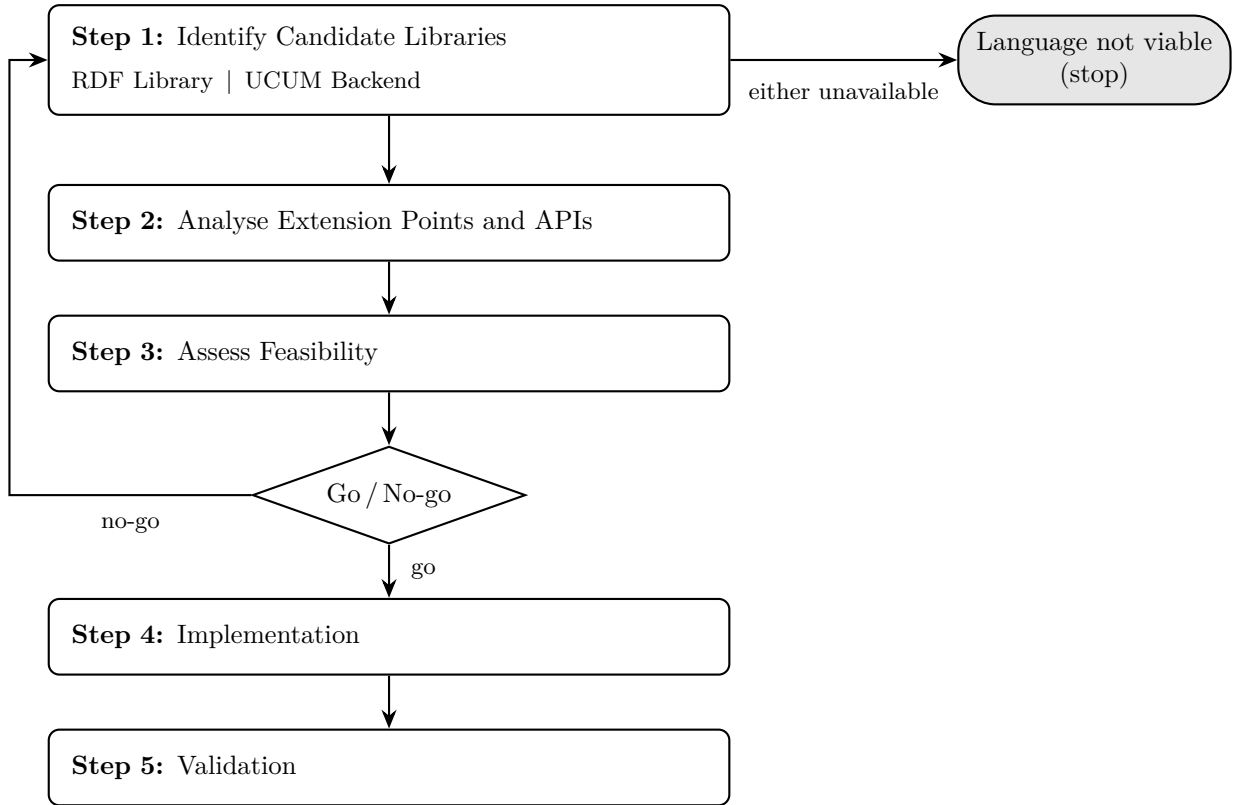


Figure 5.1: Five-step procedure with exit at Step 1 and go/no-go branch after Assess Feasibility

UCUM-native parser. When a combination is chosen, the implementer must record which library handles which responsibility, because that boundary shapes the design of the internal quantity representation; two of the four implementations required this arrangement.

The step can produce a null result on either side. If no RDF library with a custom datatype mechanism exists for the target language, or no library or pairing can parse UCUM expressions and perform arithmetic at runtime, the shortlist on that side is empty. An empty shortlist on either side means implementation is not possible and the procedure stops.

The output is a shortlist of each kind, ordered by suitability, with the reason for every elimination recorded. The ordering matters because a no-go at the feasibility step returns to the next candidate on the shortlist rather than restarting the search.

5.2 Analyze Extension Points and APIs

The goal of this step is to read the internals of both shortlisted libraries and establish how each obligation of the specification maps onto what they actually expose.

For the RDF library, the analysis covers three areas. The first is datatype registration: how the library associates a datatype IRI with a custom type, what happens at parse time when a literal carrying that IRI is encountered, whether the association can be made from outside the core, and how the extension becomes active. The second is SPARQL operator evaluation: whether each evaluation point for equality, comparison, and arithmetic is exposed for extension or hardcoded, and when hardcoded, whether the host language permits runtime intervention from outside the library; the answer determines whether a workaround is available or the obligation requires a fork. The third area is the path from SPARQL constructs to those evaluation points: whether FILTER and BIND share the same machinery as ORDER BY and the aggregates SUM and AVG, or reach separate handlers, since the answers determine whether one hook covers everything or several are needed. Finally, the implementer must check whether the library provides a function registry invocable by IRI from inside a query; the sameDimension obligation depends on it.

For the UCUM backend, the analysis covers four areas. The first is the parsing API: what it accepts and returns. The second is arithmetic: whether the backend converts between units, tracks dimensions

through operations, and signals incommensurable quantities. The third is the numeric type; a floating-point backend sets a precision ceiling no outer layer can raise. The fourth is coverage: whether all UCUM codes are supported or gaps exist.

The two cases examined here show why the analysis cannot stop at the library level. In Java, adjacent operators in the same engine received opposite answers; in Python, the operator module was reachable but ordering and aggregation held their own captured references, making an intervention appear complete while leaving those paths untouched.

The output of this step is a map from each obligation to the mechanism, if any, through which the library can satisfy it, together with the backend’s capabilities and known gaps. This map is the sole input of the feasibility assessment that follows.

5.3 Assess Feasibility

The goal of this step is to decide, before any code is written, whether the candidate pair can satisfy the specification and to record how. The assessment runs through the obligations of Section 3.4 one by one. One constraint frames the whole assessment: the implementation must be delivered as a standalone extension, ranking above every individual obligation. A fork produces a modified distribution users must run in place of the library they depend on and carry forward against every upstream release; Section 1 identified this as the shortcoming of the original Jena-ucum implementation. An obligation requiring a fork is recorded as not achievable.

For each obligation the assessment must reach one of three outcomes: satisfiable through a public API, satisfiable through a workaround, or not achievable without a fork. To reach the outcome, the implementer asks whether an extension point exists; if so, whether it is a documented public API or internals the library never promised to keep; and if not, whether the host language offers an alternative mechanism. The distinction matters because the cases age differently: a public API obligation survives upgrades, while a workaround must be re-verified at every upgrade and its interception points documented at the moment of choice.

The Python implementation shows the middle case. Comparison and arithmetic were not reachable through any public API: the engine validates operands against a fixed set of numeric types. The host language, however, permits replacing function references at runtime, so both were recorded as workarounds requiring re-verification at each library release.

When an obligation is not achievable, the implementer must either reframe it, delivering the same capability through a different interface, or return to Step 1 for the next candidate from the shortlist. The choice rests on three considerations: the centrality of the lost obligation, the availability of a viable shortlist alternative, and the library’s weight in its ecosystem. A reframing must document what it achieves and gives up against the original wording; it is a documented deviation, not a fulfilment, and that record is what keeps the conformance claim honest.

The JavaScript implementation shows how ecosystem weight settles the choice. The library carries only a minimal query layer, placing the SPARQL operator obligations out of reach, yet it remains the library its community uses.

The output of this step is a feasibility record stating, for each obligation, the case it falls under, the mechanism to be used, and any reframing adopted, together with the overall decision.

5.4 Implementation

The goal of this step is to turn the feasibility record into a working extension. The work follows a fixed order because each part depends on the one before it: the quantity representation first, registration second, the SPARQL hooks third, and the custom function fourth.

The implementer must begin with a class that wraps the UCUM backend and exposes every operation the value space requires: parsing a lexical form and writing one back, equality, comparison, arithmetic, unit conversion, and the dimension test. All backend access must pass through this class, keeping the rest of the extension independent of its API. The class must realise the value space defined in Section 3.4: two quantities in different units of the same dimension that denote the same value must compare as equal. Where the first step selected a pairing rather than a single backend, the division of labour recorded there

becomes the internal structure of this class; the parser produces what the arithmetic library consumes. With the representation in place, the implementer must register `cdt:ucum` and `cdt:ucumunit` through the extension point recorded in the feasibility record, after which the library parses a CDT literal into a typed object rather than an opaque string.

The SPARQL hooks come next. The implementer must intercept equality, comparison, and arithmetic through the mechanisms recorded per obligation, and every handler must follow the same pattern: check whether the operands are CDT literals, evaluate them through the representation if they are, and pass the call to the original behaviour unchanged if they are not. The fallback matters as much as the interception, because the extension must leave the engine's treatment of every other datatype exactly as it found it. Ill-typed literals must be caught before any value operation and routed into the engine's error path, so that the affected solution is excluded, as Section 2.3 described, rather than the whole query failing. The map from the analysis step states how many interception points this takes and where none is needed. In two implementations, registration alone already delivered SPARQL equality. In the Python case, the interception reached twenty-five points across four modules.

The custom function completes the operator work. The implementer must register `cdt:sameDimension` through the function registry recorded in the analysis. The function extracts two quantities, asks the backend whether their dimensions match, and returns a boolean.

Implementation surfaces five cases that the analysis cannot show in advance and that must be expected in any new implementation. The first is units the backend cannot parse, with the special units of Section 3.3 first among them. The second is comparison across dimensions inside ORDER BY; the query must never abort on such data, and since SPARQL leaves this ordering undefined, any deterministic grouping or fallback is conformant. The third is unit preservation in SUM, AVG, MIN and MAX, where the engine's own accumulators work on bare numbers and would return a result stripped of its unit. The fourth is precision at the numeric extremes, where the backend's internal number type sets a ceiling no outer layer can lift. The fifth is the dimensionless case: a bare number must be read as a quantity with the unit one. Each case must be handled explicitly and the chosen behaviour written down, because several will return in the next step as expected failures.

The output of this step is the working extension together with the record of edge cases and the behaviour chosen for each. That record joins the feasibility record as input to the validation step.

5.5 Validation

The goal of this step is to verify that the extension satisfies each obligation it claims and to document what it does not. The output is not a count of passing tests but an account of conformance in which every claim is backed by a test and every limitation is visible.

The implementer must first test each obligation of Section 3.4. For lexical parsing, both valid and invalid inputs must be tested: a valid UCUM expression must parse to the correct value, and an invalid one must be rejected as ill-typed. Value equality is tested across units of the same dimension, including derived units and the dimensionless case. Comparison is tested within one dimension and across dimensions, where the error required by the specification must be raised. Arithmetic is tested over its full type coverage, including scalar operands on both sides of the operator. The dimension test is checked with compatible and incompatible pairs. Each obligation must be tested at the level at which the feasibility record claims it. An obligation claimed at the query level requires tests that submit a query and inspect the results: a correct method the engine never calls passes every direct test and still fails the claim. Where an obligation was reframed, the tests target the reframed interface; no test may suggest the original form is available. The Python implementation tests at both levels: the quantity class directly, and the operators, ordering, and aggregates through queries. The Java implementation exercises equality through its interface but submits no query, so the query-level behaviour rests on the engine's documented routing rather than a test of this extension. The difference between these two positions is exactly what this step must make visible.

The implementer must then test the edge cases recorded during implementation. A query over data containing an ill-typed literal must complete, with the affected solutions excluded. Ordering over mixed dimensions must complete and follow the behaviour chosen for it. The aggregates must preserve units where the implementation claims they do. Precision is tested at scales far above and below the range of

ordinary values, where the backend’s numeric ceiling either holds or breaks. A literal written out and parsed back must keep its value. Activating the extension more than once must change nothing.

What the feasibility record marked as not achievable must now appear as expected failures. For each limitation, the implementer must write the test it causes to fail, mark it as expected, and attach the reason stating whether the cause is an inherited dependency limit or a gap in the extension itself; the two call for different responses from anyone who later builds on the work. A limitation carried this way is checked on every run and announces itself when a dependency upgrade removes it; one left out of the suite stays silent. The same two families recur across all four implementations: the special units of Section 3.3, whose conversion falls outside the algebra every backend implements, and precision loss where a backend computes in a bounded numeric type. A new implementation should expect to document both.

The output is the test suite together with the conformance account it expresses: the obligations met, the level at which each is met, the reframings, and the expected failures with their reasons. That account makes the extension’s claims checkable by someone who did not build it, and is the form in which the four implementations are reported in Section 6.

6. Implementations and Findings

This section reports the four implementations that validate the methodology of Section 5. It is organised by the five steps of that methodology rather than by library. Each section takes one step and reports how all four implementations met it, so that the section reads as a test of that step across four environments. Sections 6.1 to 6.5 follow the five steps in order. Section 6.6 collects the findings that recur across the cases.

The four implementations appear in the same fixed order throughout the section: Apache Jena in Java, rdflib.js in TypeScript, dotNetRDF in C#, and RDFLib in Python. Table 3 sets them side by side from host library to standout limitation.

Table 3: The four implementations from host to outcome. Repositories are at github.com/Irfan-Ullah-cs/.

	Apache Jena (Java)	rdflib.js (TypeScript)	dotNetRDF (C#)	RDFLib (Python)
Host library	Jena 6.0.0	rdflib.js	dotNetRDF	RDFLib 7.6.0
UCUM backend	systems-ucum + Indriya	ucum-lhc	Fhir.Metrics	ucumvert + Pint
SPARQL engine	full	none	full	full
Operator delivery	equality in query, rest reframed onto Java API	reframed onto store API	on the query surface	on the query surface
Numeric precision	fixed-precision decimal	double precision	decimal, ~28 digits	arbitrary-precision decimal
Repository	jena-ucum	rdflib.js-ucum	dotnetrdf-ucum	rdflib-ucum

Each section reports the information that determined a decision, named a mechanism, or produced a limitation. The full detail for each implementation is in the public repository.

6.1 Identify Candidate Libraries

This step looks for two things in each language: an RDF library to host the implementation and a UCUM backend to parse and compute. A short survey of each ecosystem is usually enough. RDF libraries were compared on four signals considered together: age, adoption, contributor base, and feature completeness.

None is decisive alone; a library can still be the natural choice despite lacking a major feature if the community has settled on it. On the backend side the field is narrow in every language, and the key property is whether a library can parse an arbitrary UCUM expression at runtime.

For three of the four languages the survey points to one library without much contest. In Python, RDFLib has been the general-purpose RDF library for over two decades and provides a full SPARQL 1.1 engine. In JavaScript, rdflib.js is a read-write Linked Data toolkit rather than a query engine, but its long-established community role is what recommends it, the missing engine notwithstanding. In C#, dotNetRDF has been maintained since 2009 and ships the established in-process SPARQL 1.1 engine of the .NET world.

Java is the exception, and deliberately so. The CDT specification was accompanied by a fork of Apache Jena 3 with the datatype behaviour hardcoded into the distribution. Jena was kept here to ask whether the fork's behaviour can be recovered from the outside as a standalone extension, and to move that work to the current Jena 6.0.0. Eclipse RDF4J is the other mature Java host; Jena was chosen because the prior work sat on it.

The backends were chosen on the same mixture of capability and standing. In Java the work inherited the JSR-385 pairing from the fork: systems-ucum for the UCUM grammar and unit definitions, Indriya for conversion and arithmetic. In JavaScript, ucum-lhc, from the US National Library of Medicine, covers validation, conversion, and decomposition to SI base units. In C#, Fhir.Metrics, developed by Firely for the FHIR standard, parses UCUM expressions from the official definition file loaded at runtime. No single Python library covers both; ucumvert, the only native UCUM parser, is young but generated from the official UCUM definition file, and computes through Pint, the established unit-arithmetic library of the ecosystem.

What this step cannot settle is how far each host can be reached from outside. That is the question the next section takes up.

6.2 Analyze Extension Points and APIs

This step reads the internals of each host and backend. The answers vary more between the four hosts than at any other step in the section, and that variance is the finding.

Datatype registration is the one area where every host offered a usable path, though of different kinds. RDFLib's bind ties a datatype IRI to a constructor the library calls on the lexical form at parse time. Jena's TypeMapper registry calls the registered datatype's parse method on each matching literal, marking it ill-typed if the parse throws. dotNetRDF has no registration call; recognition is achieved by subclassing the graph for the cdt:ucum IRI and adding it to the engine's recognized-types list. rdflib.js offers nothing: its literal constructor performs no dispatch on the datatype IRI, so a cdt:ucum literal is stored like any other typed string.

The SPARQL operators are where the hosts diverge most, and Jena is the clearest case because its engine treats its own operators inconsistently. Equality is delegated: ARQ routes the operator through its literal layer to the registered datatype's equality method, requiring no interception. Comparison and arithmetic are not. The less-than and greater-than operators, the four arithmetic operators, the ORDER BY comparator, and the SUM and AVG accumulators all evaluate inside engine code that consults no registry and calls no method on the datatype; compiled Java offers no stable way to intervene from outside. The split is therefore imposed by the engine: one obligation reachable through a clean public method, the rest reachable only by forking. dotNetRDF is the most accommodating: arithmetic operators go through a public registry, equality through the recognized-types list, the filter node comparer is a public settable property, and custom functions have a public factory. Only the ORDER BY comparer has no public setter; it is reachable through a single reflective access to a private field, the one non-public point in the whole extension. RDFLib exposes none of these as extension points, but Python permits replacing functions at runtime. The patching reaches further than it first appears: the operator functions live in one module, but the parser holds its own references captured when it loads, and ordering and the aggregates hold further references in separate modules; a change to the operator module alone would leave those paths running the original code while looking complete. rdflib.js has almost no SPARQL surface; its query layer handles only basic patterns and equality, comparison operators collapse to equality, ordering and aggregates are absent, there is no function registry, and the only interceptable point is a

single funnel method on the native store API.

Across the three hosts with SPARQL machinery, FILTER and BIND share one evaluation path while ordering and aggregates each require separate interception, confirming the section 5 warning against assuming one hook suffices.

All four backends parse an arbitrary UCUM expression and perform dimensional arithmetic. Three signal incompatible dimensions correctly, by an exception or a failure status the extension turns into one; the fourth does not: the C# backend treats some distinct dimensions as the same, newton and pascal among them, so an addition that should be rejected is accepted. Section 6.3 returns to this. The numeric type is where the backends part company, and the difference sets a ceiling no outer layer can lift. The Python backend can be configured to compute in arbitrary-precision decimal once a hidden fallback to floating point in the parser's transformer is corrected. The C# backend computes in the platform decimal type, roughly twenty-eight significant digits, with an overflow above that range; like the Java backend, it rounds non-terminating conversion factors to that fixed precision. The JavaScript backend computes in double-precision floating point only, as defined by IEEE 754 [23], with no higher-precision mode, so any precision beyond a double is lost before arithmetic begins. The Java backend keeps exact precision for integer-ratio conversions but rounds non-terminating conversion factors. Coverage gaps converge on the special units of Section 3.3, which the next sections report where they surface.

The step produced, for each host, a map from every obligation to the mechanism that satisfies it or the absence of one; those maps are the input to the feasibility assessment.

6.3 Assess Feasibility

The feasibility assessment read the result from the previous section against the obligations of the specification. All four candidates returned a go; what differed was what the go required.

The backend side carries one gate: the backend must parse an arbitrary UCUM expression at runtime and perform dimensional arithmetic. All four chosen backends pass it. They part company on the numeric type they compute in, and the feasibility step accepts that ceiling because no outer layer can raise it, from arbitrary-precision decimal in Python to double precision in JavaScript.

For Jena, the go required accepting that comparison and arithmetic cannot be reached from outside the library; compiled Java offers no way in without a fork. Registration, equality, and the sameDimension function are met through public APIs, so Jena still parses CDT literals, recognizes them, and answers queries that rest on parsing and equality.

For rdflib.js, the go rests on the same reasoning at a wider scale. The library has no registration mechanism and a query layer that collapses comparison operators to equality, so most operator obligations cannot be reached inside a query at all. It is still the library its community reaches for; the obligations it can meet are reframed onto the native store API, and the rest are delivered through an explicit operations layer. For dotNetRDF, every obligation is reachable on the query surface through public registries and one reflective access for the ordering comparer. One backend limitation is recorded: the Fhir.Metrics dimensional check compares unit axes without their exponents, treating some distinct dimensions as the same, newton and pascal among them, so an addition the specification requires to fail is accepted. This is a defect in the backend, not the extension, and the go is taken with it documented.

For RDFLib, the go is the most complete of the four. Every obligation is met: three through public APIs, the other two by replacing operator functions at runtime, with those replacements reversible and producing results the specification accepts. The backend computes in arbitrary-precision decimal once a single fallback in the parser is corrected.

All four are a go. dotNetRDF and RDFLib reached it with every obligation on the query surface; Jena and rdflib.js reached it by moving some off. The rule against forking the host is what separates the two pairs. Underneath all four sits the same idea: the obligations shape the decision, but a host is chosen because it is useful and used, and a partial fit on a library a community trusts is worth more than a full fit on one nobody runs.

6.4 Implementation

The four implementations followed the same fixed order as Section 5 prescribes: representation, registration, SPARQL hooks, custom function, and edge cases.

The representation class in each implementation wraps the backend and exposes parsing, equality, comparison, arithmetic, conversion, and the dimension test, realising the value space of Section 3.4; all backend access is confined to it.

Registration tied `cdt:ucum` and `cdt:ucumunit` to the representation through the mechanism named in the feasibility record, after which the host parses a CDT literal into a typed object rather than an opaque string. Where the library attaches no behaviour to a datatype IRI, recognition lives inside the extension and is applied at every point where literals enter it; `rdflib.js` works this way.

The hooks follow the same pattern in every implementation: check operands, evaluate CDT pairs through the representation, pass everything else through unchanged, and catch ill-typed literals before any value operation. What most separates the four is the number of interception points needed. In Jena and `dotNetRDF`, registration alone delivered SPARQL equality because the engine routes equality through the registered datatype.

The custom function `cdt:sameDimension` extracts two quantities, asks the backend whether their dimensions match, and returns a boolean.

The same five cases arose in every implementation: unparseable units, with the special units of Section 3.3 foremost; cross-dimension comparison in `ORDER BY`, which must never abort the query; unit stripping in `SUM` and `AVG` accumulators; precision loss at numeric extremes; and the dimensionless case, which more than one backend required a dedicated branch to handle. Each was handled explicitly and the chosen behaviour recorded, because several return in the next step as expected failures.

6.5 Validation

Before any library was tested, a single reference set of behavioral requirements was drawn up, setting out what a complete implementation should be tested for, independent of any one language. Each of the four suites is read against that same reference, and every requirement receives a status: covered, covered but behaving differently in a documented way, not applicable to the language, or absent.

The four suites differ in size, but size is not a conformance score. `RDFLib` has the most at 253 tests, none failing. `dotNetRDF` has 251, of which thirty-eight are left failing; the discussion below records which cases take which form. Jena has 163 and `rdflib.js` 157. What the totals hide, the distribution shows, because the shape of each suite tracks where its operators live. In `RDFLib` the operators run through the engine, and the query file alone holds sixty-nine tests. In `rdflib.js` the same obligations are reframed onto the store API, and the query file holds nine, with the weight moved to parsing and arithmetic on the quantity class directly. Jena has no query-level test file at all, and its largest class, sixty-seven tests, exercises the Java operations API that comparison and arithmetic were reframed onto.

Each obligation is tested at the level at which the feasibility record claims it. The Python implementation tests at both levels: the quantity class on its own, and the operators, ordering, and aggregates through queries the engine runs. The Java implementation exercises equality through the datatype interface but submits no query, so query-level equality rests on Jena's documented routing rather than on a test of the extension. Arithmetic was tested across its full type coverage; two of the four implementations were incomplete at the same point.

The edge cases from implementation were then tested: a query over data containing an ill-typed literal must complete with the affected solutions excluded; ordering over mixed dimensions must complete; aggregates must preserve units where claimed; a literal written out and parsed back must keep its value; and activating twice must change nothing.

Expected failures mark the obligations the feasibility record left as not achievable, with the cause attached to each. Two families recur across all four: the special units of Section 3.3 and precision loss in bounded numeric types. The form differs by backend. Where lower precision is the accepted behaviour, the boundary becomes a passing test that documents the loss: `rdflib.js` records a value beyond double range collapsing to infinity and the sixteenth-digit case collapsing to one float. `dotNetRDF` records all limitations as failing tests: its decimal backend overflows above roughly twenty-eight significant digits

and rounds non-terminating conversion factors; the latter is why a cross-unit equality such as 3.6 km/h against 1 m/s does not hold. The Celsius offset conversion and the Fhir.Metrics dimensional defect that reports newton and pascal as one dimension are also left as failing tests with the cause attached. In all, the dotNetRDF suite leaves thirty-eight tests failing.

6.6 Findings Across the Cases

The clearest finding is that the same obligation landed differently in each host for one reason: how far an extension could reach from outside. The host’s architecture determines this, whether operators are exposed through public points or evaluated in code an extension cannot enter, and so does the host language. A dynamic language permits replacing functions at runtime, which is how the Python operators were reached at twenty-five points without altering the library; a compiled language offers no such entry.

The rule against forking sorts the four into three positions: RDFLib and dotNetRDF with every obligation on the SPARQL surface; Jena with comparison and arithmetic reframed onto an explicit Java API while equality stays on the surface; and rdflib.js with no complete SPARQL engine. All four stay worth implementing because they are the libraries their communities use.

7. Conclusions

This thesis set out to determine whether implementing cdt:ucum in an RDF library could be turned from a one-off effort into a general procedure. It produced a five-step methodology and validated it through four standalone implementations, in Java, TypeScript, C#, and Python. Each leaves its host library unmodified, and each is tested against a single reference checklist so that the four can be read side by side. The procedure assumes no particular language and no particular library architecture, and the four cases show it holding across environments that differ in both.

The four cases also produced a finding that none of them shows alone. The obligations of the specification are fixed, but how far an extension can reach into a host from the outside is not, and that reach is what decides how each obligation gets met. It is set by two things: whether the host exposes its operators through a public point or evaluates them in code an extension cannot enter, and whether the host language permits intervention at runtime. The same specification was therefore satisfied through different mechanisms in each of the four hosts, and which mechanism applied followed from these two properties rather than from the obligations themselves.

Future Work: The clearest continuation is to apply the methodology to RDF libraries it has not yet met. RDF4J is the other major Java library; running the procedure there holds the language fixed and changes only the host architecture, isolating how much of the outcome that architecture alone decides. Sophia and rdf4cpp carry the procedure into Rust and C++, two languages absent from the four cases, where the runtime intervention that the Python implementation relied on has no obvious equivalent and the question of what reach remains is open. Each new host added this way sharpens the same map the methodology exists to draw.

References

- [1] BIPM JCGM et al. *International vocabulary of metrology - basic and general concepts and associated terms (VIM)*. 2012.
- [2] Jan Martin Keil and Sirko Schindler. “Comparison and evaluation of ontologies for units of measurement”. In: *Semantic Web 10.1* (2018), pp. 33–51.

- [3] Maxime Lefrançois and Antoine Zimmermann. “The unified code for units of measure in RDF: cdt: ucum and other UCUM datatypes”. In: *European Semantic Web Conference*. Springer, 2018, pp. 196–201.
- [4] Ralph Hodgson et al. *Quantities, units, dimensions and data types ontologies (QUDT)*. Tech. rep. Technical report, TopQuadrant and NASA, 2014.
- [5] Hajo Rijgersberg, Mark Van Assem, and Jan Top. “Ontology of units of measure and related concepts”. In: *Semantic Web 4.1* (2013), pp. 3–13.
- [6] N. J. Car et al. *OGC GeoSPARQL - A Geographic Query Language for RDF Data*. OGC Implementation Standard OGC 22-047. Version 1.1. Open Geospatial Consortium, 2024. URL: <http://www.opengis.net/doc/IS/geosparql/1.1>.
- [7] R. Cyganiak, D. Wood, and M. Lanthaler. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation. W3C, Feb. 2014. URL: <https://www.w3.org/TR/rdf11-concepts/>.
- [8] O. Hartig et al. “Datatypes for Lists and Maps in RDF Literals”. In: *Proceedings of Posters and Demos at the 21st Extended Semantic Web Conference (ESWC 2024)* (Hersonissos, Greece). May 2024, pp. 234–238. URL: <https://2024.eswc-conferences.org/wp-content/uploads/2024/05/77770226.pdf>.
- [9] S. Harris and A. Seaborne. *SPARQL 1.1 Query Language*. W3C Recommendation. W3C, Mar. 2013. URL: <https://www.w3.org/TR/sparql11-query/>.
- [10] M. Lefrançois and A. Zimmermann. *LINDT Custom Datatypes Specification*. Technical specification. École des Mines de Saint-Étienne, 2021. URL: https://w3id.org/lindt/custom_datatypes.
- [11] P. Marciniak, P. Sowiński, and M. Ganzha. “Representing and querying data tensors in RDF and SPARQL”. In: *The Semantic Web: ESWC 2025 Satellite Events* (Portorož, Slovenia). Cham: Springer, June 2025. DOI: [10.48550/arXiv.2504.19224](https://doi.org/10.48550/arXiv.2504.19224). arXiv: [2504.19224](https://arxiv.org/abs/2504.19224).
- [12] M. Perry and J. Herring. *OGC GeoSPARQL - A Geographic Query Language for RDF Data*. OGC Implementation Standard OGC 11-052r4. Open Geospatial Consortium, 2012. URL: <https://www.ogc.org/standard/geosparql/>.
- [13] G. Schadow and C. J. McDonald. *The Unified Code for Units of Measure*. Tech. rep. Regenstrief Institute. URL: <https://ucum.org/ucum>.
- [14] D. Brickley and R. V. Guha. *RDF Schema 1.1*. W3C Recommendation. W3C, Feb. 2014. URL: <https://www.w3.org/TR/rdf-schema/>.
- [15] W3C OWL Working Group. *OWL 2 Web Ontology Language: Document Overview*. W3C Recommendation. W3C, Dec. 2012. URL: <https://www.w3.org/TR/owl2-overview/>.
- [16] D. Beckett et al. *RDF 1.1 Turtle*. W3C Recommendation. W3C, Feb. 2014. URL: <https://www.w3.org/TR/turtle/>.
- [17] NEMA (National Electrical Manufacturers Association). *Digital Imaging and Communications in Medicine (DICOM), Part 16: Content Mapping Resource, Section 7.2.2 (Units of Measurement)*. URL: https://dicom.nema.org/medical/dicom/current/output/chtml/part16/sect_7.2.2.html.
- [18] C. J. McDonald et al. “LOINC, a Universal Standard for Identifying Laboratory Observations: A 5-Year Update”. In: *Clinical Chemistry* 49.4 (2003). For the Laboratory LOINC Developers, pp. 624–633. DOI: [10.1373/49.4.624](https://doi.org/10.1373/49.4.624).
- [19] HL7 International. *FHIR (Fast Healthcare Interoperability Resources) - Data Types: Quantity*. URL: <https://hl7.org/fhir/datatypes.html#Quantity>.
- [20] ISO. *Health Informatics: Identification of Medicinal Products - Data Elements and Structures for the Unique Identification and Exchange of Units of Measurement*. Tech. rep. ISO 11240:2012. International Organization for Standardization, 2012.
- [21] J. Madin et al. “An ontology for describing and synthesizing ecological observation data”. In: *Ecological Informatics* 2.3 (2007), pp. 279–296. DOI: [10.1016/j.ecoinf.2007.05.004](https://doi.org/10.1016/j.ecoinf.2007.05.004).

- [22] R. G. Raskin and M. J. Pan. “Knowledge representation in the semantic web for Earth and environmental terminology (SWEET)”. In: *Computers & Geosciences* 31.9 (2005), pp. 1119–1125. DOI: [10.1016/j.cageo.2004.12.004](https://doi.org/10.1016/j.cageo.2004.12.004).
- [23] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Tech. rep. IEEE Std 754-2019. IEEE Computer Society, 2019. DOI: [10.1109/IEEESTD.2019.8766229](https://doi.org/10.1109/IEEESTD.2019.8766229).
- [24] BD Hall and M Kuster. “Representing quantities and units in digital systems”. In: *Measurement: Sensors* 23 (2022), p. 100387.
- [25] Armin Haller et al. “The modular SSN ontology: A joint W3C and OGC standard specifying the semantics of sensors, observations, sampling, and actuation”. In: *Semantic Web* 10.1 (2018), pp. 9–32. DOI: [10.3233/SW-180320](https://doi.org/10.3233/SW-180320).